



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ
Rai Technology University
(First under Karnataka Act. No. 29 of 2013)

ASSIGNMENT

Subject Title: Deep Learning

Subject Code: OCS351T

Submitted By:

Student Name: REVANA SIDDA

University Enrolment Number: RTU24101CS048

Program Name: B tech (CSE)

Semester: 5TH SEM

Session: 2026

Submitted To

Faculty Name: Mr. Satyam Sir

Department of Computer Science & Engineering

College of Engineering

Rai Technology University Bengaluru-561204

Assignment:

Simple Neural Network:

Introduction:

Handwritten digit recognition is a common image classification problem in machine learning. The objective is to automatically identify a handwritten digit from an input image.

The MNIST dataset is widely used for evaluating image classification algorithms. It contains grayscale images of handwritten digit from 0 to 9.

Problem Statement:

to develop a neural network using tensorflow/keras that can classify handwritten digit image into one of ten classes from 0 to 9.

Objective:

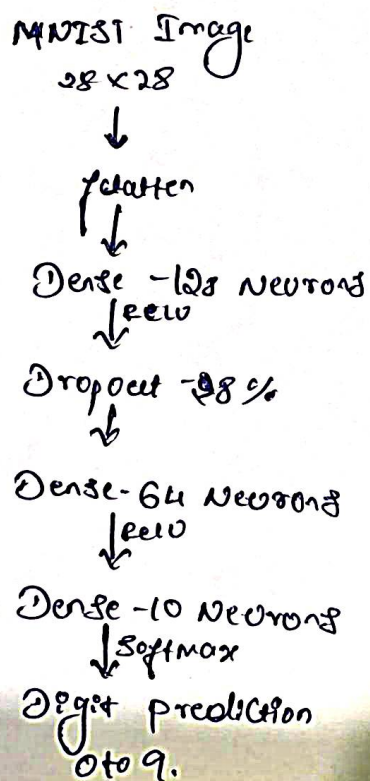
1. Load the MNIST dataset.
2. Explore the dataset.
3. Preprocess the image.
4. Build a neural network.
5. Train the model.
6. Evaluate its accuracy.
7. Visualize training performance.
8. Test five images.

9. perform an architecture experiment.

Methodology

1. load the mnist dataset.
2. Explore the dataset and display sample images.
3. Normalise pixel values.
4. flatten each 28×28 image
5. pass the flatten image through the hidden layers
6. Use the output layer containing 10 neurons.
7. train the network.
8. Evaluate it on the dataset.

Model Architecture:



Code :-

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Input, Flatten, Dense, Dropout
```

```
print("tensorflow version : ", tf.__version__)
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```

```
print("training images shape : ", x_train.shape)
```

```
print("training labels shape : ", y_train.shape)
```

```
print("testing images shape : ", x_test.shape)
```

```
print("testing labels shape : ", y_test.shape)
```

```
print("first 20 labels : ")
```

```
print(y_train[:20])
```

```
print("minimum pixel value : ", x_train.min())
```

```
print("maximum pixel value : ", x_train.max())
```

```
unique, counts = np.unique(y_train, return_counts=True)
```

```
for digit, count in zip(unique, counts):
```

```
    print(f"digit {digit} : {count} images")
```

```
plt.figure(figsize=(10,5))
```

```
for i in range(10):
```

```
    plt.subplot(5,5,i+1)
```

```
    plt.imshow(X_train[i], cmap='gray')
```

```
    plt.title("Label: %d" % y_train[i])
```

```
    plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
X_train = X_train.astype("float32") / 255.0
```

```
X_test = X_test.astype("float32") / 255.0
```

```
print("Minimum value after normalization:", X_train.min())
```

```
print("Maximum value after normalization:", X_train.max())
```

```
model = Sequential([
```

```
    Input(shape=(28,28)),
```

```
    Flatten(),
```

```
    Dense(128, activation='relu'),
```

```
    Dropout(0.2),
```

```
    Dense(64, activation='relu'),
```

```
    Dense(10, activation='softmax').
```

```
])
```

```
model.summary()
```

```
model.compile(  
    optimizer = 'adam',  
    loss = 'sparse_categorical_crossentropy',  
    metrics = ['accuracy']  
)
```

```
print("model compiled successfully.")
```

```
history = model.fit(  
    x_train,  
    y_train,  
    epochs = 10,  
    batch_size = 32,  
    validation_split = 0.1,  
    verbose = 1.  
)
```

```
test_loss, test_accuracy = model.evaluate(  
    x_test,  
    y_test,  
    verbose = 0  
)
```

```
print("Test Loss: ", test_loss)  
print("Test Accuracy: ", test_accuracy)  
print("Test Accuracy (%): ", test_accuracy * 100)
```

```
plt.figure(figsize=(8,5))  
plt.plot(history.history['accuracy'], label='training accuracy')  
plt.plot(history.history['val_accuracy'], label='validation accuracy')
```

plt.figure ('training & validation Accuracy')

plt.xlabel ('epoch')

plt.ylabel ('Accuracy')

plt.legend ()

plt.grid (true)

plt.show ()

plt.figure (figsize=(8,5))

plt.plot(history.history['loss'], label='training loss')

plt.plot(history.history['val_loss'], label='validation loss')

plt.title ('training and validation loss')

plt.xlabel ('epoch')

plt.ylabel ('loss')

plt.legend ()

plt.grid (true)

plt.show ()

predictions = model.predict(x-test)

predicted_labels = np.argmax(predictions, axis=-1)

print ('first 20 predicted labels :')

print (predicted_labels[:20])

sample_indices = [0, 1, 3, 4]

plt.figure (figsize=(12,3))

for i, index in enumerate(sample_indices):

image = x-test[index]

~~def~~

actual = y_test[index]

predicted = predicted_labels[index]

plt.subplot(1, 5, i+1)

plt.imshow(image, cmap='gray')

plt.title(f"Actual = {actual} | Predicted = {predicted}")

plt.axis('off')

plt.figure_layout()

plt.show()

print("Actual vs predicted labels")

print("-" * 35)

for index in sample_indices:

actual = y_test[index]

predicted = predicted_labels[index]

result = "Correct" if actual == predicted else "Incorrect"

print(

f"Image {index + 1}:"

f"Actual = {actual},"

f"Predicted = {predicted}"

f"Result = {result}"

```
Experiment-Model = sequential([  
    Input (Shape = (28, 28)).
```

```
    Flatten().
```

```
    Dense (256, activation = 'relu').
```

```
    Dropout (0.2).
```

```
    Dense (128, activation = 'relu'),
```

```
    Dense (10, activation = 'softmax')
```

3)

```
Experiment-Model.compile(  
    optimizer = 'adam':
```

```
    loss = 'Sparse-Categorical-Crossentropy':
```

```
    metric = 'Accuracy')  
)
```

```
Experiment-History = Experiment-Model.fit(  
    x = train,
```

```
    y = train,
```

```
    epochs = 10,
```

```
    batch-size = 32,
```

```
    validation-split = 0.1
```

```
    verbose = 1  
)
```

```
Experiment-Loss, Experiment-Accuracy = Experiment-Model.evaluateevaluate(  
    x = test,
```

```
    y = test,
```

```
    verbose = 0.  
)
```

```
Print ("Experimental model test loss :", experiment_loss)
```

```
Print (  
    "Experimental model test Accuracy:",  
    experiment_accuracy.
```

```
)
```

```
Print (  
    "Experimental model test Accuracy (%):",  
    Experiment - accuracy * 100
```

```
)
```

Conclusion:

The MNIST handwritten digit classification project successfully demonstrated the use of tensorflow/keras to build image classification.